

Code-Layout, Readability and Reusability

Date	01 JULY 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	yes	
2	Proper File Structure	Files and folders are logically organized	yes	
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	
4	Function / Method Names	Functions are descriptively named	Yes	
5	Code Comments	Inline and block comments explain logic	C Partial	
6	Modular Design	Code is split into reusable functions/modules	Yes	
7	No Redundant Code	Duplicate or unused code is removed	Yes	
8	Error Handling	Exceptions and errors are handled gracefully	Cefully Yes	

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	where Reused	Reusability Level (High / Medium / Low)
1	Data Preprocessing & Encoding Pipeline (src/data_preprocessing.py)	Python / scikit-learn (Pipeline, ColumnTransformer, StandardScaler, OneHotEncoder, SimpleImputer), joblib	Fits the preprocessing steps during training (	

	src/data_preprocessing.py)		build_assets.py) and is saved as a serialized pipeline to be reused in real-time inference (app.py).	High
2	Model Inference Flask API Web Server (app.py)	Python / Flask, pandas, joblib	Acts as the core user-facing web app, running prediction inference, rendering templates (index.html and result.html), and serving health check endpoints.	High
3	Hyperparameter Tuning & Benchmarking Harness (src/train_models.py)	Python / scikit-learn (GridSearchCV, StratifiedKFold), xgboost	Reused by developers to run and compare multiple algorithms (Logistic Regression, Decision Tree, Random Forest, XGBoost) and log comparison metrics.	High
4	Dataset Downloader & Setup Utility (src/utls.py)	Python / urllib.request, os, logging	Reused in the main production assets build pipeline (build_assets.py) to fetch raw datasets on fresh startup or deployment.	Medium
5	Exploratory Data Analysis (EDA) Engine (src/eda.py)	Python / pandas, seaborn, matplotlib	Run as an offline analysis script; the generated visualizations (distributions, correlations, rates) are saved directly to static/images/plots/ and served dynamically on the dashboard UI.	Medium
6	Target Label Engineering & Merging Pipeline (src/data_pipeline.py)	Python / pandas	Reused in the main build script (build_assets.py) to clean duplicates and engineer risk labels from raw records.	Medium
7	Champion Model Production Trainer (src/train_champion.py)	Python / scikit-learn (RandomForestClassifier), joblib	Reused in the deployment build pipeline (build_assets.py) to quickly train the selected champion model with optimized hyperparameters and serialize it for prediction.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Excellent separation of concerns. Core ML logic is organized in the <code>src/</code> folder, while web logic is kept in the root <code>app.py</code> . Assets, raw/processed datasets, templates, and models are isolated in logical, Git-ignored folders. The build sequence is beautifully orchestrated from <code>build_assets.py</code> .
Readability	4.5	Highly readable. Standard Python and ML libraries (pandas, scikit-learn) are used idiomatic to industry conventions. Modular functions are kept concise and well-named. Readability could be further elevated by adding PEP-484 type hints to function signatures.
Reusability	4.5	The pipeline serialization strategy—reusing the fitted <code>preprocess_pipeline</code> model preprocessor directly in the real-time Flask <code>predict</code> routing logic—is highly professional and prevents train/test skew. Reusability could be slightly improved by moving hardcoded files, URLs, and column names into a configuration file.
Documentation / Comments	5	Exceptional documentation. Functions have clear docstrings explaining inputs and outputs. Crucial domain-specific decisions—such as what constitutes a default/bad payment status, why certain ID columns are dropped to prevent leakage, and how class imbalance is handled—are thoroughly explained in inline comments.
Overall Score	4.8	Production-grade codebase. The project successfully establishes a complete end-to-end Machine Learning pipeline (data fetching, engineering, validation splits, benchmarking, training, and deployment) with clean, clean patterns throughout.